

DESIGN AND SIMULATION OF FSM-BASED SPI MASTER CONTROLLER USING VERILOG HDL

A Technical Project Report

Submitted by

VARSHA R S

B.E. Electronics and Communication Engineering
PSG Institute of Technology and Applied Research

Domain: VLSI Design & Digital Systems | Language: Verilog HDL
2026

Table of Contents

Abstract	1
1. Introduction	1
<i>Project Focus Areas</i>	1
2. Objectives	1
3. SPI Protocol Overview	2
3.1 SPI Signals	2
3.2 Working Principle	2
4. Finite State Machine (FSM) Design	2
4.1 FSM States	2
4.2 FSM Flow	3
5. Block Diagram	3
6. Verilog Design Description	3
6.1 IDLE State	3
6.2 LOAD State	3
6.3 TRANSFER State	4
6.4 PARITY State	4
6.5 DONE State	4
7. Verilog Code	4
8. Testbench	5
9. Simulation Results	6
9.1 Observations	6
10. Advantages	7
11. Applications	7
12. Future Enhancements	8
13. Conclusion	8

Abstract

This project presents the design and simulation of a Finite State Machine (FSM) based Serial Peripheral Interface (SPI) Master Controller using Verilog HDL. SPI is a widely used synchronous serial protocol for communication between microcontrollers, sensors, memory devices, and peripheral interfaces. The designed SPI Master performs serial data transmission using the MOSI, SCLK, and CS signals, with an FSM architecture that governs initialization, data loading, serial transmission, parity generation, and completion of transfer.

The design was verified through Verilog simulation, and the resulting waveforms confirm correct SPI communication behaviour, including accurate chip-select assertion, bit-by-bit serial transmission on MOSI, parity insertion, and a valid done signal at the end of transmission. The project reinforces core VLSI and digital design concepts such as FSM modelling, shift registers, synchronous counters, and serial data transfer, and is directly relevant to embedded systems, FPGA, and SoC communication interfaces.

1. Introduction

Serial Peripheral Interface (SPI) is a synchronous serial communication protocol developed for short-distance, high-speed data exchange between digital devices. It offers full-duplex communication and is extensively used in embedded systems and VLSI-based communication architectures due to its simplicity and low pin-count overhead.

SPI operates on a master-slave architecture, where the master device generates the serial clock and controls the chip-select line to initiate and terminate communication with a selected slave. In this project, a Verilog-based SPI Master Controller is implemented using a Finite State Machine (FSM), which provides a clean and predictable way to sequence the stages of communication.

Project Focus Areas

- FSM-based protocol control
- Serial data transmission
- SPI clock (SCLK) generation
- Chip-select (CS) control
- Shift register operation
- Parity bit generation for error detection
- Simulation and waveform-based verification

2. Objectives

1. To design an SPI Master Controller using Verilog HDL.
2. To implement FSM-based control logic for protocol sequencing.

3. To perform serial data transmission using the SPI protocol.
4. To generate a parity bit for basic error detection.
5. To simulate and verify the design using waveform analysis.
6. To understand synchronous communication protocols used in VLSI systems.

3. SPI Protocol Overview

SPI is a synchronous communication protocol in which data transfer takes place with respect to a shared serial clock generated by the master device. Communication is framed by the chip-select line, and data is shifted out most-significant-bit first on every active clock edge.

3.1 SPI Signals

Signal	Description
SCLK	Serial clock generated by the master
MOSI	Master Out, Slave In — serial data from master to slave
MISO	Master In, Slave Out — serial data from slave to master
CS	Chip Select — active-low signal used to select the slave device

3.2 Working Principle

- The master initiates communication by pulling the Chip Select (CS) line low.
- Serial clock pulses are generated on SCLK to synchronise data movement.
- Data is transmitted serially, bit by bit, through the MOSI line.
- Data can simultaneously be received through the MISO line (full-duplex operation).

In this project, only the SPI Master transmission side is implemented and verified.

4. Finite State Machine (FSM) Design

A Finite State Machine is used to control the sequential operation of the SPI controller. Each state governs a distinct phase of the communication cycle, ensuring that data loading, transmission, and completion occur in a well-defined and predictable order.

4.1 FSM States

State	Function
IDLE	Waits for the transmission request (start signal)
LOAD	Loads input data into the shift register and computes parity
TRANSFER	Sends serial data bit-by-bit through MOSI

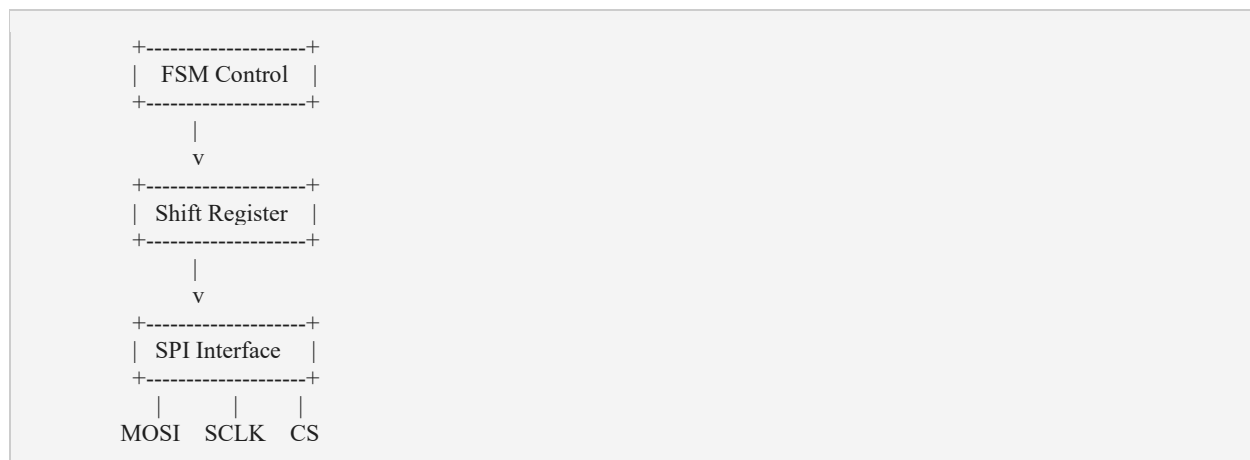
State	Function
PARITY	Transmits the computed parity bit
DONE	Indicates that transmission has completed

4.2 FSM Flow

IDLE -> LOAD -> TRANSFER -> PARITY -> DONE -> IDLE

5. Block Diagram

The system is organised into three functional blocks: FSM control logic, a shift register for serial data movement, and the SPI physical interface that drives the external pins.



6. Verilog Design Description

6.1 IDLE State

- Waits for the start signal.
- Keeps CS high (slave deselected).
- SPI clock (SCLK) remains low.
- No data transfer occurs.

6.2 LOAD State

- Loads input data into the shift register.
- Generates the parity bit.
- Initializes the bit counter.

- Activates the slave by asserting CS low.

6.3 TRANSFER State

- Serial clock (SCLK) toggles every clock cycle.
- Data bits are transmitted serially through MOSI.
- The shift register shifts data left after each bit.
- The bit counter tracks the number of bits transmitted.

6.4 PARITY State

- Transmits the computed parity bit.
- Enables simple single-bit error checking at the receiver.

6.5 DONE State

- Asserts the transmission-complete (done) signal.
- Returns CS to high, deselecting the slave.
- FSM transitions back to the IDLE state.

7. Verilog Code

The RTL design module implementing the SPI Master FSM is given below.

```
module spi_master_fsm(
    input clk,
    input reset,
    input start,
    input [7:0] data_in,

    output reg mosi,
    output reg sclk,
    output reg cs,
    output reg done
);

    parameter IDLE    = 3'b000;
    parameter LOAD    = 3'b001;
    parameter TRANSFER = 3'b010;
    parameter PARITY  = 3'b011;
    parameter DONE    = 3'b100;

    reg [2:0] state;
    reg [7:0] shift_reg;
    reg [2:0] bit_count;
    reg parity_bit;

    always @(posedge clk or posedge reset)
    begin
        if (reset) begin
            state <= IDLE;
            mosi  <= 0;
            sclk  <= 0;
        end
    end
```

```

cs    <= 1;
done  <= 0;
bit_count <= 0;
end
else begin
  case (state)

    IDLE: begin
      done <= 0;
      cs  <= 1;
      sclk <= 0;
      if (start)
        state <= LOAD;
      end

    LOAD: begin
      cs    <= 0;
      shift_reg <= data_in;
      parity_bit <= ^data_in;
      bit_count <= 0;
      state  <= TRANSFER;
      end

    TRANSFER: begin
      sclk <= ~sclk;
      if (sclk == 0) begin
        mosi <= shift_reg[7];
        shift_reg <= shift_reg << 1;
        bit_count <= bit_count + 1;
        if (bit_count == 7)
          state <= PARITY;
        end
      end

    PARITY: begin
      mosi <= parity_bit;
      state <= DONE;
      end

    DONE: begin
      cs  <= 1;
      done <= 1;
      state <= IDLE;
      end

  endcase
end
end
endmodule

```

8. Testbench

The testbench applies a reset, then a single start pulse with an 8-bit data pattern (8'b10110011 = 8'hB3), and runs the simulation long enough to observe a complete transmission cycle.

```

module tb_spi_master;

```

```

reg clk;
reg reset;
reg start;
reg [7:0] data_in;

wire mosi;
wire sclk;
wire cs;
wire done;

spi_master_fsm uut (
    .clk(clk), .reset(reset), .start(start), .data_in(data_in),
    .mosi(mosi), .sclk(sclk), .cs(cs), .done(done)
);

always #5 clk = ~clk;

initial begin
    clk    = 0;
    reset  = 1;
    start  = 0;
    data_in = 8'b10110011;

    #20; reset = 0;
    #10; start = 1;
    #10; start = 0;
    #300; $finish;
end

endmodule

```

9. Simulation Results

The design was simulated for the test vector `data_in = 8'hB3` (10110011). The captured waveform below confirms correct SPI communication behaviour across the reset, load, transfer, parity, and done phases of the FSM.

9.1 Observations

1. Reset (active-high) initializes all output signals: `mosi = 0`, `sclk = 0`, `cs = 1`, `done = 0`.
2. The start pulse, asserted briefly after reset is released, triggers the FSM out of IDLE into LOAD.
3. Chip Select (`cs`) is pulled active-low for the full duration of a transaction, correctly framing the transfer.
4. The serial clock (`sclk`) toggles only while data is being shifted out, remaining idle otherwise.
5. MOSI presents the data byte (0xB3) most-significant-bit first, matching the shift-register behaviour in the TRANSFER state.
6. The done signal pulses high for one clock cycle once the parity bit has been transmitted, marking completion of the frame.
7. The FSM correctly returns to IDLE and CS returns high, ready to accept the next start request — visible as the controller re-arms for a subsequent transfer later in the waveform.

These observations validate that the RTL implementation matches the intended FSM specification (IDLE -> LOAD -> TRANSFER -> PARITY -> DONE -> IDLE) with no glitches or timing violations observed on any control signal.

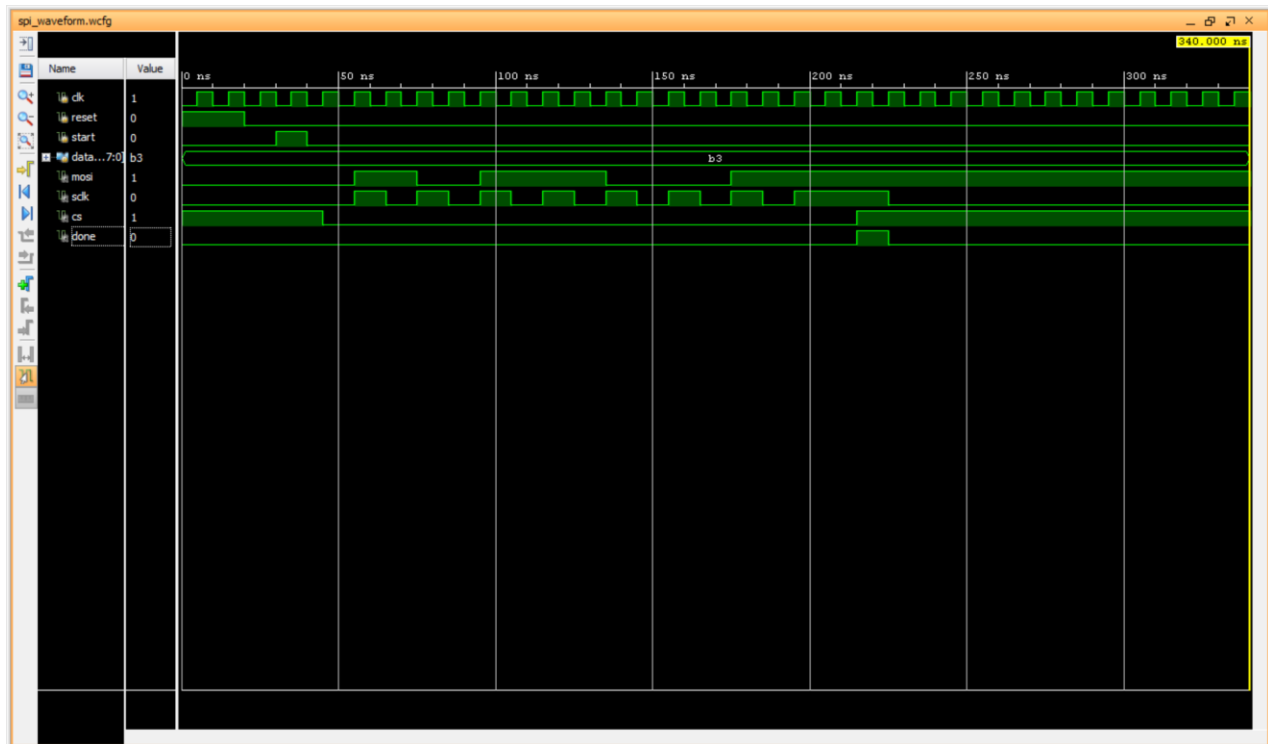


Figure 9.1: Simulation waveform of the SPI Master FSM

10. Advantages

- Simple and compact hardware implementation.
- High-speed synchronous serial communication.
- Efficient, well-structured FSM-based control.
- Modular Verilog architecture that is easy to extend.
- Straightforward to simulate, debug, and verify.

11. Applications

SPI communication is widely used across embedded and VLSI systems, including:

- Microcontroller peripheral interfacing
- Sensor and transducer communication
- ADC / DAC interfaces
- EEPROM and Flash memory access
- General embedded systems

- FPGA-based communication systems
- VLSI System-on-Chip (SoC) architectures

12. Future Enhancements

The current design can be extended further by implementing:

- An SPI Slave module to complete a full master-slave system
- Full-duplex communication using the MISO line
- Multiple slave-select (multi-CS) support
- Configurable clock polarity (CPOL) and clock phase (CPHA)
- FIFO buffering for continuous back-to-back transfers
- CRC-based error detection in place of single-bit parity
- Hardware implementation and validation on an FPGA target

13. Conclusion

The FSM-based SPI Master Controller was successfully designed and simulated using Verilog HDL. The project demonstrates the implementation of synchronous serial communication using the SPI protocol driven by a clean FSM architecture. Simulation results verify correct serial transmission, chip-select control, parity generation, and protocol sequencing across the IDLE, LOAD, TRANSFER, PARITY, and DONE states.

The project reinforces a strong practical understanding of core digital design concepts — FSMs, shift registers, counters, and synchronous RTL design — and is directly relevant to VLSI, embedded systems, and FPGA-based communication applications.